VISION ANNOTATION AUDIT

Portfolio implementation report | Ahmad Essam Abdelaziz | 7 November 2026

Status: Working MVP published to GitHub on a dedicated feature branch. Pull request #1 is open and unmerged. All 20 local tests passed and GitHub Actions Python CI succeeded.

20 / 20	8 findings	CI passed
Automated tests	Corrupt fixture errors	GitHub Actions

1. What the application does

A lightweight, offline quality gate for COCO object-detection annotation files. It finds structural and geometry problems before model training: duplicate IDs, missing image/category references, impossible or out-of-bounds bounding boxes, suspicious area discrepancies, duplicate boxes and unused categories. Target users: CV students, ML engineers and dataset curators.

2. Research and ranked project selection

Official employer postings from Simbe Robotics, Zoox, Amazon and Graphcore provided evidence of demand for Python engineering, data quality, quantitative validation, test automation and clear technical reporting. These postings are skill-demand signals, not proof of the user's eligibility.

Rank	Project idea	Editorial score
1	Vision Annotation Audit	91 / 100
2	SQL Query Plan Explainer	86 / 100
3	Incident Log Correlator	84 / 100
4	Model Inference Profiler	81 / 100

Rubric (100 points): demand 25, portfolio gap 20, usefulness 15, differentiation 15, feasibility 15 and cost 10. Scores are editorial judgments, not measured hiring probabilities. This project adds computer-vision dataset auditing without duplicating the existing reliability, model-drift or API-contract projects.

3. Architecture and source files

COCO JSON → parser → schema/reference/geometry checks → deterministic issue records → JSON and Markdown reports → CI-compatible exit code.

- src/vision_audit/core.py — reusable audit functions and issue generation
- src/vision_audit/__main__.py — command-line interface, I/O and status codes
- examples/ — clearly synthetic clean/corrupt fixtures and expected reports
- tests/test_core.py — 20 automated tests; .github/workflows/ci.yml — Python CI
- MASTER_PROMPT.md — implementation specification, boundaries and acceptance criteria

4. Install and run — step by step

Clone the repository or use the source ZIP. On a standard Python 3.10+ environment, run these commands from the repository root:

```
python -m pip install -e .
python -m vision_audit examples/clean.json --json-out clean-report.json --md-out clean-report.md
python -m vision_audit examples/corrupt.json --json-out bad-report.json --md-out bad-report.md
python -m unittest discover -s tests -v
```

CLI exit codes: 0 means no validation errors, 2 means invalid annotations, and 3 means an input/output or JSON error. Warnings alone do not fail the gate.

5. Reproduced examples and actual verification

Operation	Observed result
Unit test suite	20 tests passed
Python compilation	Passed
Clean synthetic sample	1 image, 1 annotation, 1 category; 0 errors, 0 warnings; exit 0
Corrupt synthetic sample	2 image records, 3 annotation records, 2 categories; 8 errors, 3 warnings; exit 2
Remote GitHub Actions	Python CI completed successfully (run 42370345652)

6. What is and is not verified

This is a functional offline CLI, tested against synthetic examples, not a production dataset certification service. It does not read image pixels, check real image-file existence, validate polygons/keypoints/segmentation or train any ML model. COCO area can represent segmented-object area, so an area-to-box disagreement is a warning. Repeated identical boxes may sometimes be legitimate. Docker, public deployment and real-world accuracy benchmarks are not claimed.

7. GitHub publication — verified

- Repository: github.com/Ahmad-Joker/vision-annotation-audit
- Branch: feat/vision-annotation-audit-mvp
- Pull request: PR #1, open and not merged
- GitHub Actions: Python CI run 42370345652 — success

Verified source commit: 93e23725269b10602dc7ebf350e8f6d7cad2d0ce. Subsequent documentation edits may create additional commits and CI runs. No merge or deployment has been performed.

8. How to explain the project in an interview

“I built a deterministic COCO annotation-quality gate to catch dataset defects before CV training. It checks references, IDs and bounding-box geometry, produces machine-readable and human-readable reports, and uses exit codes so CI can reject invalid datasets. I separated core validation logic from the CLI, wrote 20 tests including deliberately corrupt synthetic fixtures, and validated the GitHub Actions pipeline.”

9. Truthful CV bullet

Developed an offline Python COCO annotation auditor with 20 passing tests and a successful GitHub Actions pipeline; added deterministic JSON/Markdown reports and CI-ready exit codes to detect duplicate IDs, broken references and invalid bounding boxes before model training.

10. Master prompt and future improvements

The saved MASTER_PROMPT.md defines the target users, schema and geometry rules, report formats, CLI exit codes, synthetic fixtures, edge cases, automated tests, architecture, documentation and publication checkpoints. Implementation followed a build/test/inspect/fix loop. Suggested next improvements: optional image-file verification, support for segmentation/keypoints, user-configurable warning thresholds, and a dataset visualization view.

11. Primary hiring sources and competing products

- Simbe Robotics — CV intern
- Zoox — data science intern
- Amazon — ML SDE intern
- Graphcore — QA graduate

Existing products such as CVAT, FiftyOne and Datumaro cover more comprehensive dataset operations. Vision Annotation Audit deliberately focuses on a dependency-light and transparent CLI quality gate; it is not a novel research algorithm.

Publication verified 7 November 2026. Source and PDF are preserved in the project GitHub branch and the ChatGPT Library.